

ISC Assignment-05
Jeetu Mehra
U24AI032

Question 1 : Write a python program to implement the Hill Cipher for encryption and decryption of messages. A ground control station needs to securely transmit a short tactical command to a field drone using a 2 x 2 Hill Cipher. The program should construct and display the key matrix, format the plaintext into valid 2-letter blocks, perform encryption and decryption, and verify the result.

Scenario:

A ground control station needs to securely transmit a short tactical command to a field drone using a 2 x 2 Hill Cipher.

Key Matrix: $K = \begin{bmatrix} 3 & 3 \\ 2 & 5 \end{bmatrix}$

- **Plaintext:** MEET AT BASE

Tasks - Write a program to:

1. Construct and display the 2 x 2 Hill Cipher key matrix.
2. Convert the plaintext to uppercase, remove spaces, and map letters to numbers (A=0, B=1, ..., Z=25).
3. Divide the plaintext into valid 2-letter blocks (pad with 'X' if needed).
4. Encrypt each 2-letter block using the Hill Cipher formula: $C = K * P \pmod{26}$.
5. Display the encrypted numerical blocks and the corresponding ciphertext.
6. Find the modular inverse of the key matrix modulo 26.
7. Decrypt the ciphertext using the inverse key matrix: $P = K^{-1} * C \pmod{26}$.
8. Convert decrypted numerical values back to letters and verify the original plaintext is recovered.

Code:

```
# AI 331 - Information Security and Cryptography
# Lab Assignment 5
# Hill Cipher

import numpy as np

def get_key_matrix():
    # 2x2 Key Matrix K = [[3, 3], [2, 5]]
    K = np.array([[3, 3],
                  [2, 5]])

    return K

def show_key_matrix(K):
    print("\n2 x 2 Key Matrix:")
    for row in K:
```

```

        print(" ".join(str(x) for x in row))

def text_to_numbers(text):
    text = text.upper().replace(" ", "")
    nums = [ord(ch) - ord('A') for ch in text if ch.isalpha()]
    return text.replace(" ", ""), nums

def make_blocks(nums):
    # Pad with 'X' (23) if odd length
    if len(nums) % 2 != 0:
        nums.append(ord('X') - ord('A'))
    blocks = []
    for i in range(0, len(nums), 2):
        blocks.append([nums[i], nums[i + 1]])
    return blocks

def mod_inverse(a, m):
    # Find modular inverse of a mod m
    for x in range(1, m):
        if (a * x) % m == 1:
            return x
    return None

def matrix_mod_inverse(K, m=26):
    det = int(round(np.linalg.det(K))) % m
    det = det % m
    det_inv = mod_inverse(det, m)

    if det_inv is None:
        print("Key matrix is not invertible mod 26!")
        return None

    # Adjugate of 2x2 matrix
    adj = np.array([[K[1][1], -K[0][1]],
                    [-K[1][0], K[0][0]]])

    inv = (det_inv * adj) % m
    return inv.astype(int)

def encrypt(blocks, K):
    enc_blocks = []
    for block in blocks:
        vec = np.array(block)
        result = K.dot(vec) % 26
        enc_blocks.append(result.tolist())
    return enc_blocks

def decrypt(enc_blocks, K_inv):
    dec_blocks = []

```

```

        for block in enc_blocks:
            vec = np.array(block)
            result = K_inv.dot(vec) % 26
            dec_blocks.append(result.tolist())
        return dec_blocks

def blocks_to_text(blocks):
    text = ""
    for block in blocks:
        for num in block:
            text += chr(int(num) + ord('A'))
    return text

# ----- MAIN PROGRAM -----

print("==== HILL CIPHER ====")

# 1. Key Matrix
K = get_key_matrix()
show_key_matrix(K)

# 2. Plaintext
plaintext = "MEET AT BASE"
print("\nPlaintext:", plaintext)

# Convert to numbers
clean_text, nums = text_to_numbers(plaintext)
print("Cleaned Text:", clean_text)
print("Numerical Values:", nums)

# 3. Make 2-letter blocks
blocks = make_blocks(nums)
print("\nPlaintext Blocks:", [clean_text[i*2:i*2+2] for i in range(len(blocks))])

# 4 & 5. Encrypt
enc_blocks = encrypt(blocks, K)
print("\nEncrypted Numerical Blocks:", enc_blocks)
ciphertext = blocks_to_text(enc_blocks)
print("Ciphertext:", ciphertext)

# 6. Modular Inverse of Key Matrix
K_inv = matrix_mod_inverse(K)
print("\nModular Inverse of Key Matrix (mod 26):")
for row in K_inv:
    print(" ".join(str(x) for x in row))

# 7 & 8. Decrypt and verify
dec_blocks = decrypt(enc_blocks, K_inv)
decrypted = blocks_to_text(dec_blocks)
print("\nDecrypted Text:", decrypted)
print("Original (cleaned):", clean_text)

```

```
print("Verified:", decrypted == clean_text or decrypted.startswith(clean_text))
```

Output:-

```
PS d:\PROGRAMMING\programms\ISC> cd "d:/PROGRAMMING/programms/ISC/LAB-5" ; python Q1.py

===== HILL CIPHER =====

2 x 2 Key Matrix:
3 3
2 5

Plaintext: MEET AT BASE
Cleaned Text: MEETATBASE
Numerical Values: [12, 4, 4, 19, 0, 19, 1, 0, 18, 4]

Plaintext Blocks: ['ME', 'ET', 'AT', 'BA', 'SE']

Encrypted Numerical Blocks: [[22, 18], [17, 25], [5, 17], [3, 2], [14, 4]]
Ciphertext: WSRZFRDCOE

Modular Inverse of Key Matrix (mod 26):
15 17
20 9

Decrypted Text: MEETATBASE
Original (cleaned): MEETATBASE
Verified: True
```